

# HTML Forms with Node.js

# 02

Hey, Techna, we've now built the website, but now the problem is collecting volunteers' and donors' information. Is there a way we can systematically collect and store that data?

Yes...HTML forms are the best way to do this. Let me show you how to do it.

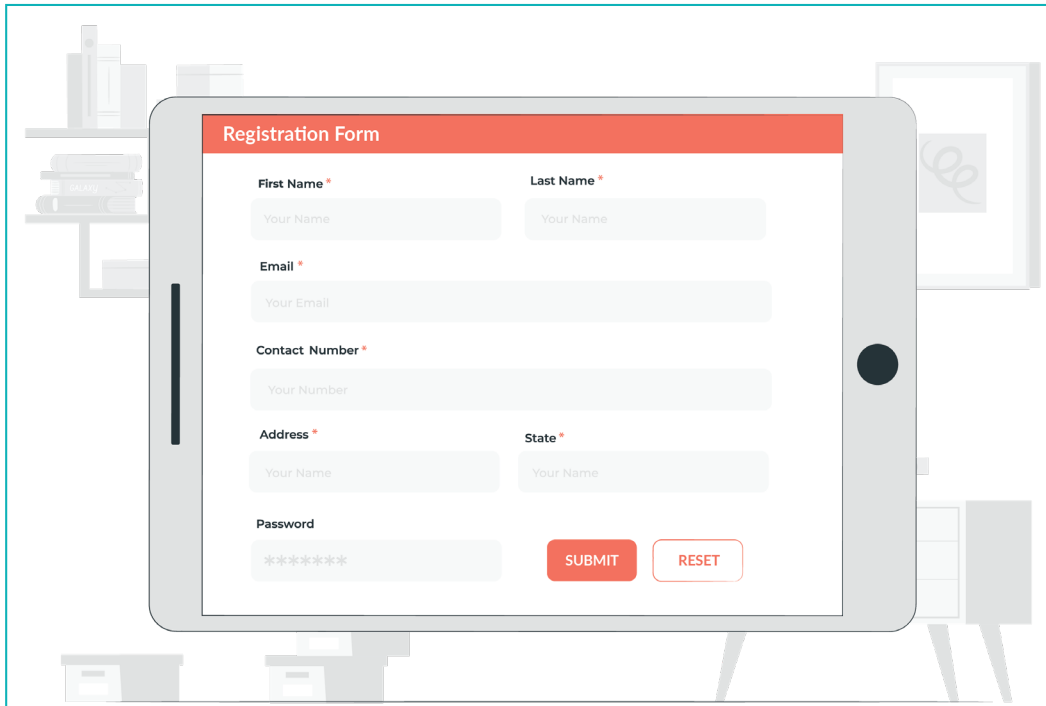


In the previous lesson, we got familiar with Node.js and its basic setup. We built a website while learning about things like module installation and importing using npm, routing using static Files, etc.

## What will we learn in this lesson?

In this lesson, we are going to create an HTML Form and save the user data in a JSON file. Let us get started with creating the HTML form first.

### What is HTML Form?



The image shows a tablet displaying a registration form. The form has a red header bar with the text 'Registration Form'. Below the header, there are several input fields: 'First Name \*' and 'Last Name \*' (both with placeholder text 'Your Name'), 'Email \*' (with placeholder text 'Your Email'), 'Contact Number \*' (with placeholder text 'Your Number'), 'Address \*' and 'State \*' (both with placeholder text 'Your Name'), and 'Password' (with placeholder text '\*\*\*\*\*'). At the bottom right of the form, there are two buttons: 'SUBMIT' and 'RESET'.

We have all used an HTML form, whether knowingly or unknowingly, to sign up, log in, or provide a shipping address on a website. HTML Forms are an excellent way to collect information from site visitors. For instance, if a person wishes to volunteer, assist, or donate to our Animal Welfare organization, they must fill out a form providing their full name, email address, and bank account information for donations.

Similarly, when a company conducts a survey to solicit employee feedback on formal issues, they use the HTML form.

Likewise, when applying for different positions in a company or admitting a patient to a hospital, all the basic details are collected by them via an HTML form.

Furthermore, while conducting research, all data required by the researchers is circulated among many people in the form of a link and collected only in HTML form.

There are numerous day-to-day instances where people prefer using HTML forms for data collection and storing them in a systematic format.

An HTML form consists of various form elements, such as textboxes, dropdown lists, checkboxes, buttons, etc., through which a user can provide their details and then submit it to be stored on the website's server.

#### Note:

This lesson is a follow-up to Lesson 1, and we will not create a new Node project. We will continue working on the project that we started in Lesson 1.

### Activity

In this lesson, we will create a form where a user can provide their name, email, address, and phone number.

#### Step 1: Creating the HTML Page

- To define our Form, we must first create a view. So, head over to the 'views' folder, create a new file called "Register.html" and add the basic HTML boilerplate code to it.



The screenshot shows the VS Code interface. On the left, the Explorer pane displays a file tree with folders like 'node\_modules', 'public', 'css', 'images', and 'views'. The 'views' folder is expanded, showing 'Home.html' and 'Register.html'. 'Register.html' is selected and highlighted with a yellow box. On the right, the editor shows the content of 'Register.html'. The code is basic HTML boilerplate, with lines 1 through 13. A yellow box highlights the entire code block.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Register</title>
8 </head>
9 <body>
10
11 </body>
12 </html>
13
```

- Now, create a section and define a heading for the Form. Add a class "form\_section" to the section as well, as shown in the figure.



The screenshot shows the VS Code editor with the 'Register.html' file open. The code is the same as the previous screenshot, but with additional lines. A red box highlights a new section added to the body. The code is as follows:

```
7   <title>Register</title>
8 </head>
9 <body>
10
11   <section class="form_section">
12     <h2> JOIN US </h2>
13   </section>
14
15 </body>
16 </html>
```

### Activity

#### Step 2: Rendering the Register.html Page

Before proceeding with the rest of the page, we will render this page so that it can be viewed in the browser.

- 1 Open 'index.js' and define a route for this page. For the path, we will specify a route '/register', but you can specify a different route path if you prefer.

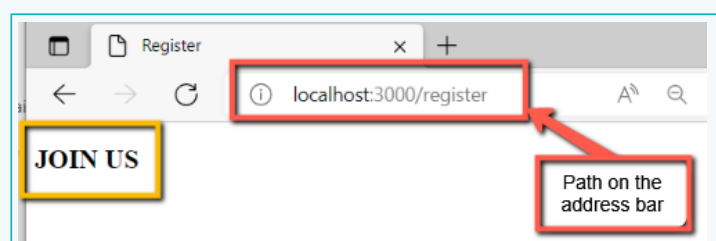
```
7 app.get('/', function(req,res) {  
8   res.sendFile('D://AnimalWelfare/views/Home.html')  
9 })  
10  
11 app.get('/register', function(req,res) {  
12   res.sendFile('D://AnimalWelfare/views/Register.html')  
13 })  
14  
15 app.listen(3000, function() {
```

- 2 To view this page in the browser, first launch your application by opening the terminal and executing the command 'node index'.

```
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Try the new cross-platform PowerShell https://aka.ms/pscore6  
  
PS D:\AnimalWelfare> node index  
Node Server is running  
█
```

- 3 Now, open your preferred browser, and in the address bar, type "localhost:3000/register".

- 4 Note that '/register' refers to the route for the register.html page.



As can be seen, the contents of register.html are now displayed in the browser.

### Activity

#### Step 3: Creating the HTML Form

Now that our page is up and running, we can proceed to create the Registration Form.

- 1 To create a form, we will use the **<form>** tag. This **<form>** tag will contain all the form elements that we will use.

```
7      <title>Register</title>
8    </head>
9    <body>
10     <section class="form_section">
11       <h2> JOIN US </h2>
12
13       <form>
14
15       </form>
16
17     </section>
18   </body>
19 </html>
```

The first form element component comprises a 'label' and a 'text box' that allow the user to provide his/her name.

- 2 To create a label, we will use the **<label>** element/tag and for creating the text box, we will use the **<input>** element/tag with the 'type' attribute, set to text.

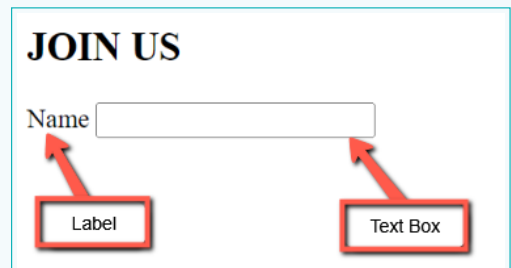
```
register.html / ...
<section class="form_section">
  <h2> JOIN US </h2>
  <form>
    <label> Name </label>
    <input type="text" name="name" required>
  </form>
</section>
```

### Activity

- 3 Additionally, we will use two more attributes called 'name' and 'required'. The 'name' attribute specifies a name for the HTML element, and the 'required' attribute specifies that the input element must be filled out before the form submission.

```
11 <h2> JOIN US </h2>
12 <form>
13   <label> Name </label>
14   <input type="text" name="username" required>
15
16 </form>
17 </section>
18
19
```

- 4 Let us check the output on the browser.



- 5 We will add a few more text boxes where the user can provide their 'age', 'phone number', and 'email id'. For age and phone number, we will set 'type' to 'text' and for email, we will set it to 'email'. Setting the type to 'email' ensures that the input value is a properly formatted email address.

- 6 Add appropriate 'name' and 'required' attributes to each form element.

```
<label> Name </label>
<input type="text" name="username" required>

<label> Age </label>
<input type="text" name="age" required>

<label> Phone Number </label>
<input type="text" name="phone" required>

<label> Email </label>
<input type="email" name="email" required>

</form>
```

### Exercise

Fill in the blanks.

- 1 To create a text-box input field, change the 'type' attribute to \_\_\_\_\_.
- 2 The \_\_\_\_\_ attribute ensures that an input field is filled out before Form submission.

### Activity

- 7 Next, we will use the **<textarea>** element to create an input field for the user's address.
- 8 The **<textarea>** element generates a multi-line input field suitable for addresses, comments, or descriptions. This field requires two additional attributes, namely 'rows' and 'cols' to define the height and width of the text field.

```
<label> Email </label>
<input type="email" name="email" required>

<label> Address </label>
<textarea name="address" cols="30" rows="5"> </textarea>

</form>
</section>
```

- 9 Finally, we will create a 'submit' button using the **<button>** element and setting the 'type' attribute to 'submit'.

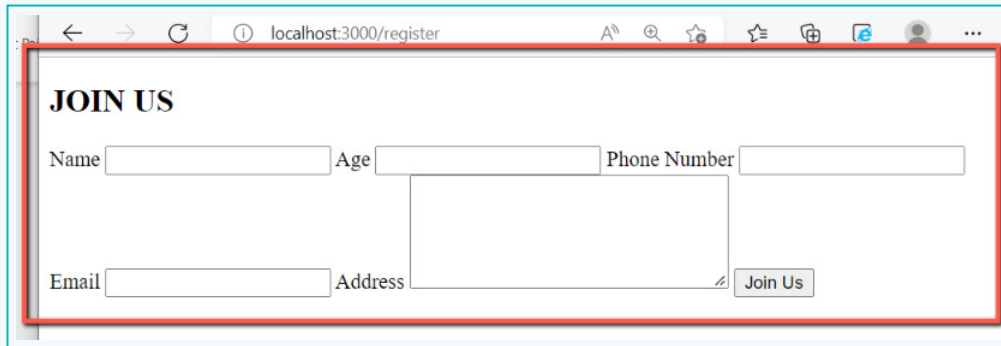
```
<label> Address </label>
<textarea name="address" cols="30" rows="5"> </textarea>

<button type="submit"> Join Us </button>

</form>
</section>
```

### Activity

- 10 This is how the final registration form looks after all the fields are filled in.



- 11 We can see from the form that the form elements are present, but they appear unorganised.
- 12 This is clearly because we have not yet styled them.
- 13 So, before proceeding, we will design the form and enhance its appearance.

### Exercise

Tick mark ☒ the correct answer

- 3 Which HTML element can be used to create a multi-line input field?

|                               |                          |                                  |                          |
|-------------------------------|--------------------------|----------------------------------|--------------------------|
| a. <code>&lt;input&gt;</code> | <input type="checkbox"/> | b. <code>&lt;textarea&gt;</code> | <input type="checkbox"/> |
| c. <code>&lt;radio&gt;</code> | <input type="checkbox"/> | d. <code>&lt;select&gt;</code>   | <input type="checkbox"/> |

- 4 Which HTML attribute does not belong to the `<textarea>` element?

|         |                          |         |                          |
|---------|--------------------------|---------|--------------------------|
| a. Type | <input type="checkbox"/> | b. Rows | <input type="checkbox"/> |
| c. Cols | <input type="checkbox"/> | d. Name | <input type="checkbox"/> |



### Activity

#### Step 4: Designing the Form using CSS

In the previous lesson, we created the 'style.css' file, and we will use it again to define our new CSS properties.

- 1 First, we will link our 'style.css' file with 'Register.html' using the `<link>` tag.

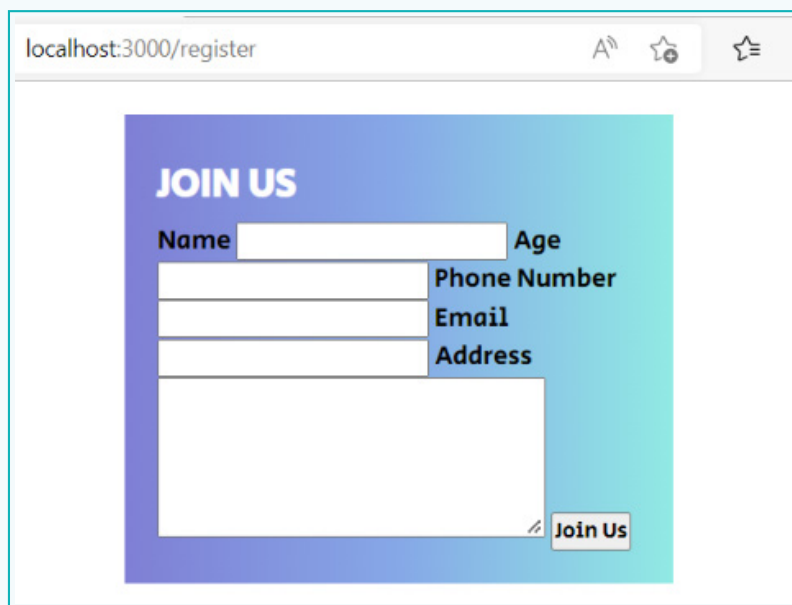
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Register</title>
8   <link rel="stylesheet" href="css/style.css">
9
10
11 </head>
```

- 2 Now, open style.css and add the following CSS properties.

```
<> Register.html # style.css X
public > css > # style.css > ...
65 .form_section {
66   width: 35%;
67   margin: auto;
68   margin-top: 20px;
69   background: linear-gradient(to right, #7f7fd5, #86a8e7, #91eae4);
70   padding: 20px;
71 }
72
73 .form_section h2 {
74   padding: 5px 0;
75   color: white;
76   border-radius: 5px;
77 }
78
```

### Activity

- 3 HTML Form after applying the CSS properties.

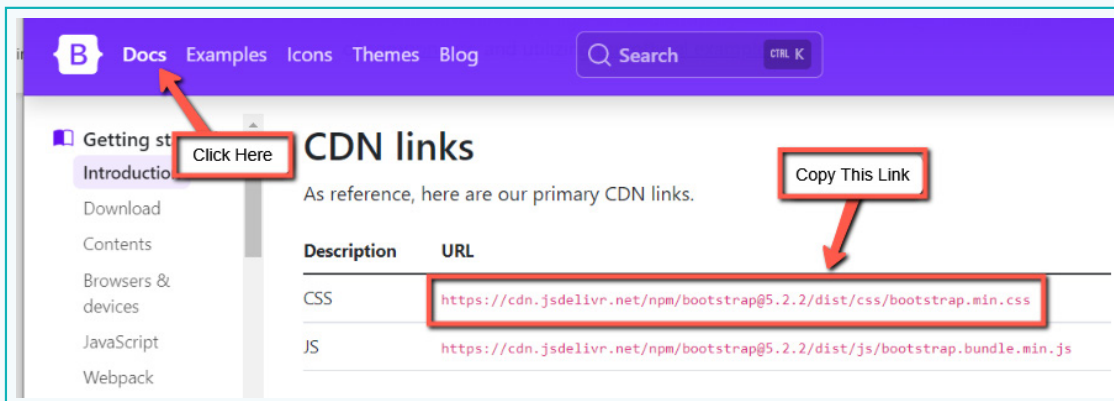


The screenshot shows a web browser window with the address bar displaying 'localhost:3000/register'. The main content area features a registration form with a blue gradient background. The form is titled 'JOIN US' in white text. It contains several input fields: 'Name' (a single line), 'Age' (a single line), 'Phone Number' (a single line), 'Email' (a single line), and 'Address' (a multi-line text area). A 'Join Us' button is located at the bottom right of the form. The form is centered on the page.

- We used CSS to reduce the width of our form, align it in the centre of the page, and add a gradient background, along with several other basic CSS properties.
- Moreover, as can be seen, our form is not completely designed. The Form elements have not yet been designed. To design Form elements, we will use Bootstrap.
- Keep in mind that it is perfectly acceptable to design our Form elements using CSS. We chose Bootstrap because it is simpler and requires less code.
- Bootstrap is a comprehensive framework that includes a variety of pre-built designs for typography, forms, tables, navigation, and layouts, among other things. It is available at no cost, has a user-friendly interface, and requires less coding.
- The simplest way to get started with Bootstrap is to head over to their official website and copy the CDN Links available there and use it on our page.

### Activity

- 4 On Bootstrap's official website, [www.getbootstrap.com](http://www.getbootstrap.com), click on the 'Docs' tab, and scroll down to where it displays 'CDN Links'. Copy the provided CSS link.



- 5 On our Register.html page, create a new 'link' tag within it.

- The 'link' tag is used when we want to link an external resource to our HTML page. This tag necessitates the use of an 'href' attribute, which specifies the link of the external resource.

- 6 In this case, we will use the 'CDN' link we just copied as the 'href' attribute value.

```
Register.html X
views > <> Register.html > html > body > section.form_section > form > label
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Register</title>
8   <link rel="stylesheet" href="css/style.css">
9
10  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.2/dist/css/bootstrap.min.css">
11
12 </head>
```

### Exercise

State if the statements are True or False

5 The <link> is placed inside the <head> tag.

▲ \_\_\_\_\_

6 The src attribute belongs to the <link> tag.

▲ \_\_\_\_\_

### Activity

#### Step 5: Designing the Form Elements using Bootstrap

Bootstrap offers various classes for designing Form elements. From designing 'Form Labels' to 'Buttons', Bootstrap has pre-defined classes for all. To use Bootstrap, simply use the 'Bootstrap' classes as values for the 'class' attribute.

1 In Register.html, use the 'form-label' class for the <label> element and 'form-control' class for the <input> and <textarea> elements, as shown below:

```
<form>
  <label class="form-label"> Name </label>
  <input type="text" name="username" class="form-control" required>

  <label class="form-label"> Age </label>
  <input type="text" name="age" class="form-control" required>

  <label class="form-label"> Phone Number </label>
  <input type="text" name="phone" class="form-control" required>

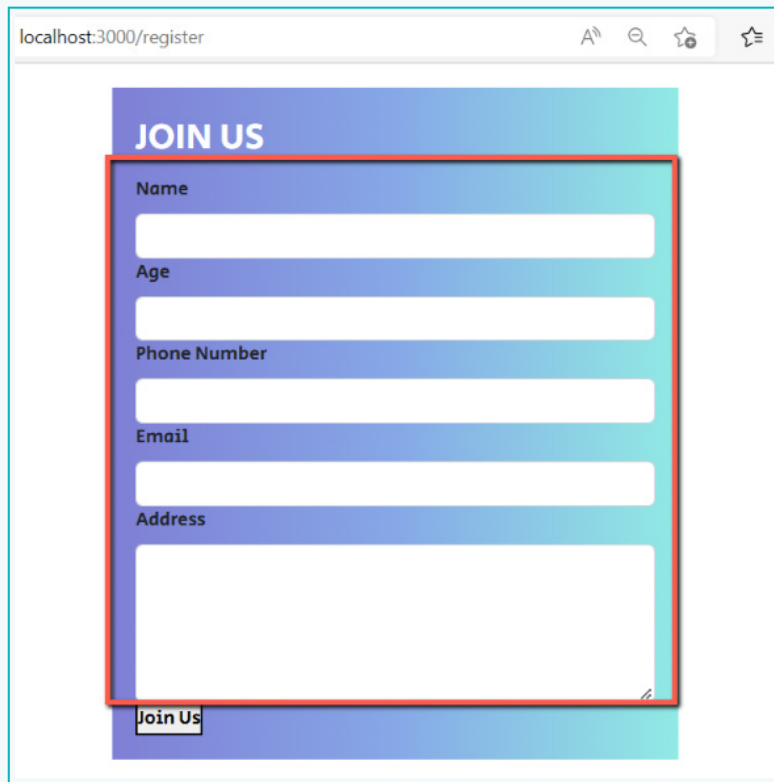
  <label class="form-label"> Email </label>
  <input type="email" name="email" class="form-control" required>

  <label class="form-label"> Address </label>
  <textarea name="address" cols="30" rows="5" class="form-control" required> </textarea>

  <button type="submit"> Join Us </button>
```

### Activity

- 2 After applying the Bootstrap classes, we can see that the Form's elements are well-organized, and no complicated code was required.



- 3 For buttons, Bootstrap offers a different class, as shown in the figure.

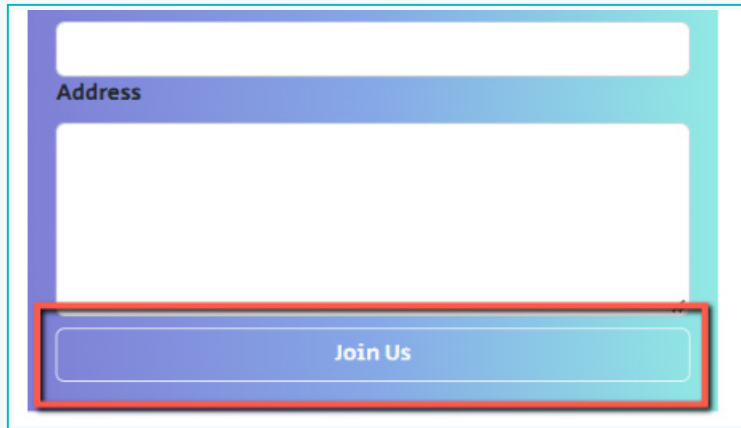
```
<label class="form-label"> Address </label>
<textarea name="description" id="" cols="30" rows="5" class="form-control"></textarea>
<button type="submit" class="btn btn-outline-light w-100 mt-3"> Join Us </button>
</form>
```

#### Class Information -

- **btn btn-outline-light:** This Bootstrap class applies some basic styles to buttons.
- **w-100:** This Bootstrap class makes the button wider to cover the entire width.
- **mt-3:** This Bootstrap class applies margin at the top.

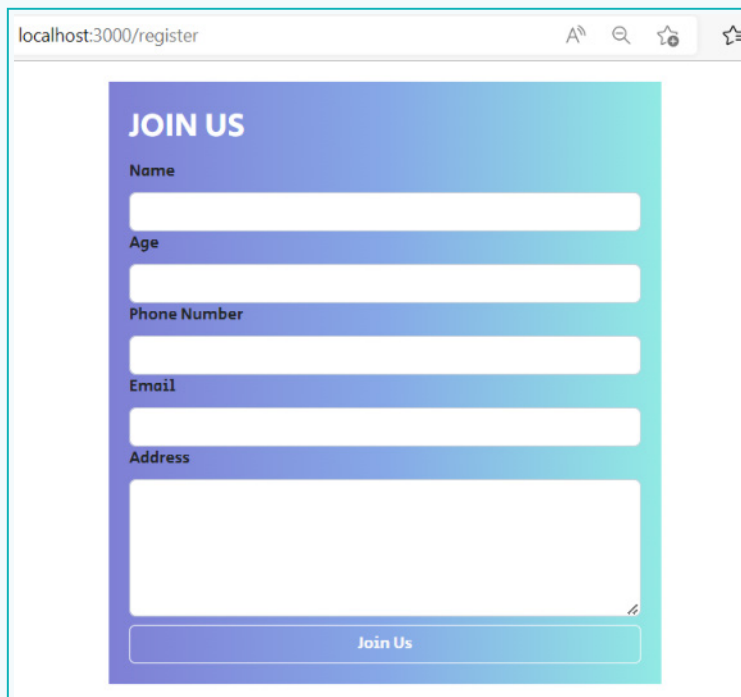
### Activity

- 4 This is how the button looks after the Bootstrap classes have been applied.



The image shows a close-up of a button with a blue gradient and rounded corners. The button is labeled "Join Us" in white text. It is highlighted by a red rectangle.

- 5 This is how the final Form looks like.



The image shows a screenshot of a web browser displaying a registration form titled "JOIN US". The form has a blue header and a light blue body. It contains five input fields: "Name", "Age", "Phone Number", "Email", and "Address". Each field is preceded by its label. At the bottom of the form is a "Join Us" button. The browser's address bar shows "localhost:3000/register".

At this point, we have completed creating and designing the Form. Next, we will learn how to retrieve user input values from the Form fields and save them in a 'JSON' file.

### What is JSON?

- When a user fills out the Form fields and submits the form, we need to define the functionality to permanently store their information somewhere.
- Usually, data is stored in a Database, but the concept of databases is not covered in this particular lesson.
- Instead, we will store the user data in a JSON-encoded text file.
- **JSON**, which stands for 'JavaScript Object Notation', is a file format that is used for storing and transmitting data to and from web applications.
- Data is stored in a 'JSON' file in 'key-value' pairs, which makes it easy to use and understand.
- In the following examples 1 and 2 store data in JSON format: The 'title' and 'author' are keys, and 'Alice in Wonderland' and 'Lewis Carroll' are 'values'.
- Each 'key-value' pair is separated by a comma.
- Also, when storing data, each key and its corresponding value must be enclosed in double-quotes.
- Double-quotes are omitted only when the values are 'Integers' or 'Booleans'.

#### Note:

**Boolean** represents a value that can only have two values – 'True' or 'False'.

For example, storing information such as 'Is your age greater than 18 or not' can be considered 'Boolean'.

```
{  
  "title": "Alice in Wonderland",  
  "author": "Lewis Carroll "  
}
```

Fig: Example 1

```
{  
  "Country_name": "India",  
  "Population(in crores)": 138  
}
```

Fig: Example 2

- The preceding example describes a JSON object, and everything inside the curly brackets belongs to that single object. In a JSON file, data is always stored in the form of an object.

#### Note:

It is important to remember that **JSON** is not a programming language and cannot be used to build websites or applications.

### Exercise

#### ■ Fill in the blanks.

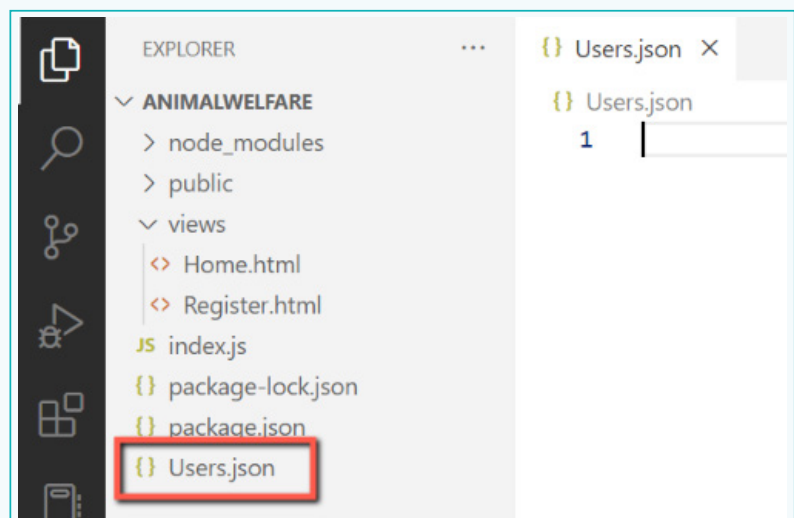
- 7 JSON stands for \_\_\_\_\_.
- 8 Double Quotes are omitted if the value we want to store is either a \_\_\_\_\_ or \_\_\_\_\_.

### Activity

#### ■ Step 6: Creating a JSON file

To store the data, first, we need to create a JSON file.

- 1 Inside our project folder, create a file "Users.json".



- 2 We will not be inserting any data manually into our JSON file.
- 3 Whatever data we are supposed to store in this file will be stored automatically whenever a user clicks on the 'Join Us' button on the Registration Form.
- 4 We will be implementing this functionality in the next steps.



### Activity

#### Step 7: Working with the Form Data

To store the user input form values, we need to first define the mechanism to obtain those data. So, let us configure that.

- 1 To handle HTML Forms form data, we must first use the 'express.urlencoded()' function with the extended parameter set to 'true'. So, open the 'index.js' file and copy the following code into it:

**app.use(express.urlencoded({extended: true}))**

```
JS index.js > ...
1  const express = require('express')
2  const app = express()
3
4  app.use(express.static('public'));
5
6  app.use(express.urlencoded({extended: true}));
7
8  app.get('/', function(req,res) {
```

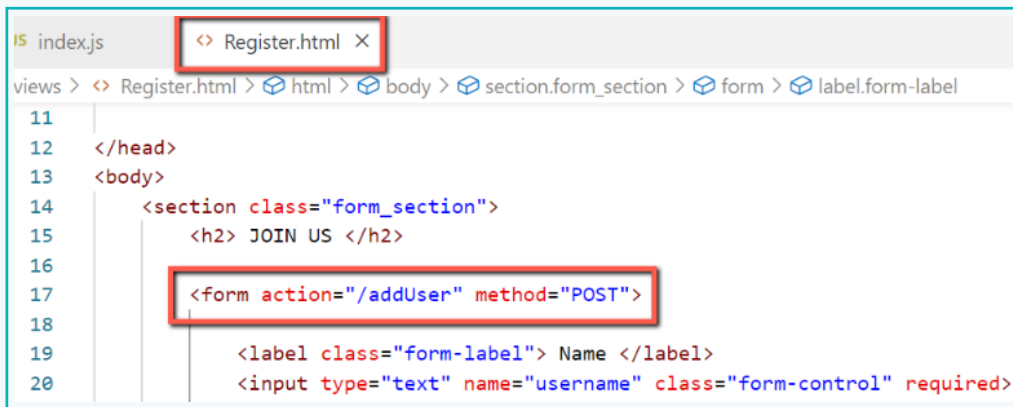
Our Node Application is now capable of handling and processing the incoming form data. Next, we must define a route that will be in charge of processing it.

- 2 In the 'index.js' file, create a route to handle the form data. It is important to emphasize that we must use the 'app.post()' method instead of 'app.get()' in this case. It is important to remember that whenever a route is supposed to receive data (like from an HTML Form), we should use the 'app.post' method.

```
12  app.get('/register', function(req,res) {
13    res.sendFile('D://AnimalWelfare/views/Register.html')
14  })
15
16  app.post('/addUser', function(req, res) {
17
18  })
19
20  app.listen(3000, function() {
21    console.log("Node Server is running")
22  })
```

### Activity

- 3 Add this route to the Registration Form's 'action' attribute.
- 4 This is done so that the '/addUser' route is executed when the 'Join Us' button on the Form is clicked.
- 5 Also, set the 'method' attribute's value to 'POST.'



```
15 index.js
16 Register.html
17 views > <> Register.html > html > body > section.form_section > form > label.form-label
18 11
19 12 </head>
20 13 <body>
21 14 <section class="form_section">
22 15 <h2> JOIN US </h2>
23 16
24 17 <form action="/addUser" method="POST">
25 18
26 19 <label class="form-label"> Name </label>
27 20 <input type="text" name="username" class="form-control" required>
```

- 6 Now, in the 'addUser' route's 'callback' function, create an object "newUser". The purpose of creating this object is to store the user information obtained from the Registration Form.



```
16 app.post('/addUser', function(req, res) {
17
18   const newUser = {
19     name: req.body.username,
20     age: req.body.age,
21     phone: req.body.phone,
22     email: req.body.email,
23     address: req.body.address
24   };
25
26 })
```

Objects always store data in 'key: value' pairs. In the object 'newUser', we have defined the following keys – 'name', 'age', 'phone', 'email', and 'address'. For each of these keys, we are storing the corresponding values received from 'Register.html'.

Here, using the 'req.body' property, we access the 'Form Elements' based on their respective names and store the values in the appropriate keys. For example, 'req.body.username' refers to the name input field in the Form, 'req.body.age' refers to the age field in the Form, and so on.

### Activity

The JavaScript object 'newUser' that we created stores all the data received from the Registration Form. To store it in the 'Users.json' file, we need to first convert this object into 'JSON' format. This is easily accomplished by using an in-built function 'stringify()'. The 'stringify()' method is used to convert any JavaScript value to JSON format.

- 7 Use the 'stringify()' method, passing the object 'newUser' as a parameter. Save the converted JSON object in a variable 'jsonContent'.

```
18 app.post('/addUser', function(req, res) {  
19     const newUser = {  
20         name: req.body.username,  
21         age: req.body.age,  
22         phone: req.body.phone,  
23         email: req.body.email,  
24         address: req.body.address  
25     };  
26     var jsonContent = JSON.stringify(newUser);  
27  
28 }
```

### Exercise

Tick mark ☒ the correct answer

- 9 Which of the following attributes belongs to the **<form>** tag?
- |           |                          |         |                          |
|-----------|--------------------------|---------|--------------------------|
| a. action | <input type="checkbox"/> | b. type | <input type="checkbox"/> |
| c. href   | <input type="checkbox"/> | d. src  | <input type="checkbox"/> |
- 10 While defining an object, which type of bracket is used?
- |        |                          |        |                          |
|--------|--------------------------|--------|--------------------------|
| a. ( ) | <input type="checkbox"/> | b. { } | <input type="checkbox"/> |
| c. [ ] | <input type="checkbox"/> | d. < > | <input type="checkbox"/> |

### Activity

#### Step 8: Storing Form Data on Users.json File

Before we can learn about how to store data on a JSON file, it is essential to have a basic understanding of the 'fs' module in Node.js.

When working on a Node project, in certain cases, we may need to read or write data to/from a file. This file can be a simple text file, a JSON file, a CSV file, and so on. For example, in our application, we have a JSON file where we need to write(store) user information.

In such cases, we require a module called the 'fs' Module. However, before we get into the 'fs' Module, let us get familiar with 'Modules'.

**Modules** are blocks of pre-defined JavaScript code that perform/execute a specific task. These are stored in either a single file or a group of files and are primarily used to add new features or enhance the existing functionality of an application.

**Node.js** includes several in-built modules, each of which performs a specific function. A few of such modules, along with their functionality, are listed below.

- **'http' module** - Creates an HTTP Server to allow Node applications to transfer data across a network.
- **'fs' module** - Handles file-related tasks, such as reading from files, writing, deleting files, and so on.
- **'os' module** - Contains information about the Operating System.

To use a module in a Node Application, we use the 'require' function.

#### The Syntax for using a module -

```
const <module> = require('module_name')
```

For example, to use the 'fs' module, we can import it as follows -

```
const fs = require('fs')
```

In our application, we will make use of the 'fs' module available in the 'Node.js module' library. This built-in module in Node.js, enables us to create, read, write, update, and delete files on our computer.

Now that we are familiar with the 'fs' module, let us look at how to write data to a JSON file.

- 1 First, let us begin by importing the 'fs' module. Open 'index.js' and import the module using the 'require()' method.

```
JS index.js > app.get('/') callback
1  const express = require('express')
2  const app = express()
3
4  const fs = require('fs')
5
6  app.use(express.static('public'));
```

### Activity

Next, we will again navigate back inside the `/addUser` route. Now that we have the form data converted into JSON format, we will use the `writeFile()` method from the `fs` module to write it to the `Users.json` file.

#### About the `writeFile()` Method

This is a simple method for writing specified data to a file. Two important points to remember while using this method are -

- If the file does not already exist, the `writeFile()` method creates a new one and writes the data to it.
- If the file already exists and contains data, the `writeFile()` method replaces it with the new data.

The syntax of this method is -

**`fs.writeFile(file_name, content, callback_function)`**

- 2 Now, let us use the `writeFile()` method to write the form data to the `Users.json` file.

```
28     var jsonContent = JSON.stringify(newUser);
29
30     fs.writeFile('Users.json', jsonContent, function(err) {
31         if(err) {
32             console.log('Error')
33         }
34         else {
35             res.redirect('/');
36         }
37     })
38 }
```

Code Explanation –

- **Users.json:** This is the file where we want to store/write the user data.
- **jsonContent:** This is the variable that contains the JavaScript object, converted into JSON format.
- **function(err) { ... }:** This is a callback function that will be called once the `writeFile()` method is executed. In this function, we define an 'if-else' condition that checks for any potential errors. If an error occurs, it will display the message "Error" on the console; otherwise, it will redirect to the root (`/`) route.

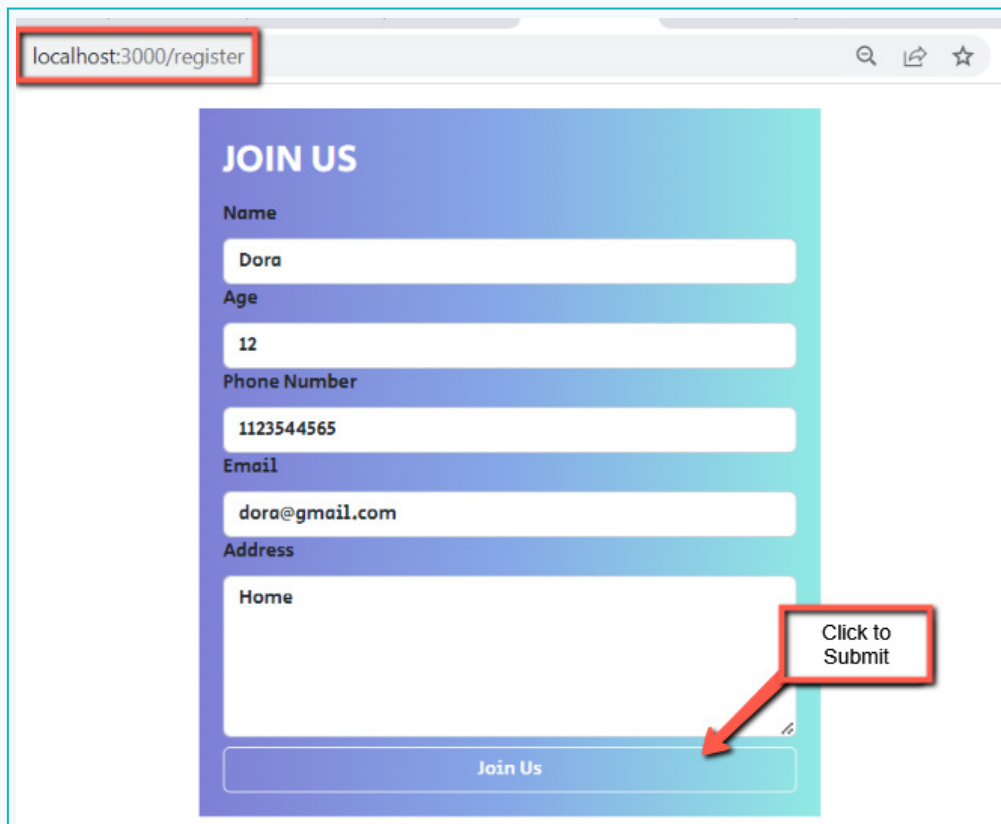
### Exercise

■ Fill in the blanks.

- 11 The 'writeFile()' method belongs to the \_\_\_\_\_ module.
- 12 To import the 'fs' module, we use the \_\_\_\_\_ function.

### Activity

- 3 Now, we can test the functionality by visiting the route "localhost:3000/register" to open the Registration Form. Fill out the Form with some values and submit it by clicking on the 'Join Us' button.



localhost:3000/register

### JOIN US

Name  
Dora

Age  
12

Phone Number  
1123544565

Email  
dora@gmail.com

Address  
Home

Join Us

Click to Submit

### Activity

- 4 If everything works fine, you will be redirected to the 'Home' page, and the values you have entered on the Form will be available on the 'Users.json' file as shown below-

```
{ } Users.json •
{ } Users.json 2:
1 { "name": "Dora", "age": "12", "phone": "1123544565", "email": "dora@gmail.com", "address": "Home " }
2
3
```

Notice that the data in the 'Users.json' file is not stored in a randomized order, rather it is properly formatted in 'key: value' pairs, with each 'key:value' pair separated by a comma.

As a result, randomized content is stored in a properly formatted manner, making it much easier to access and identify the relevant values, which are properly separated by a comma.

This is how our website will look:

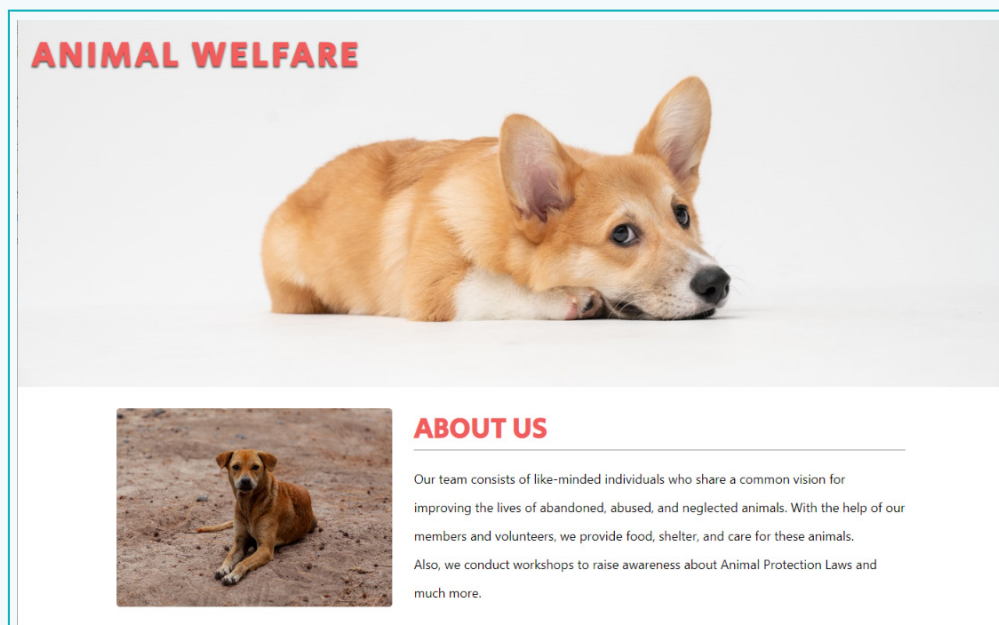


Figure: Home Page

### Activity

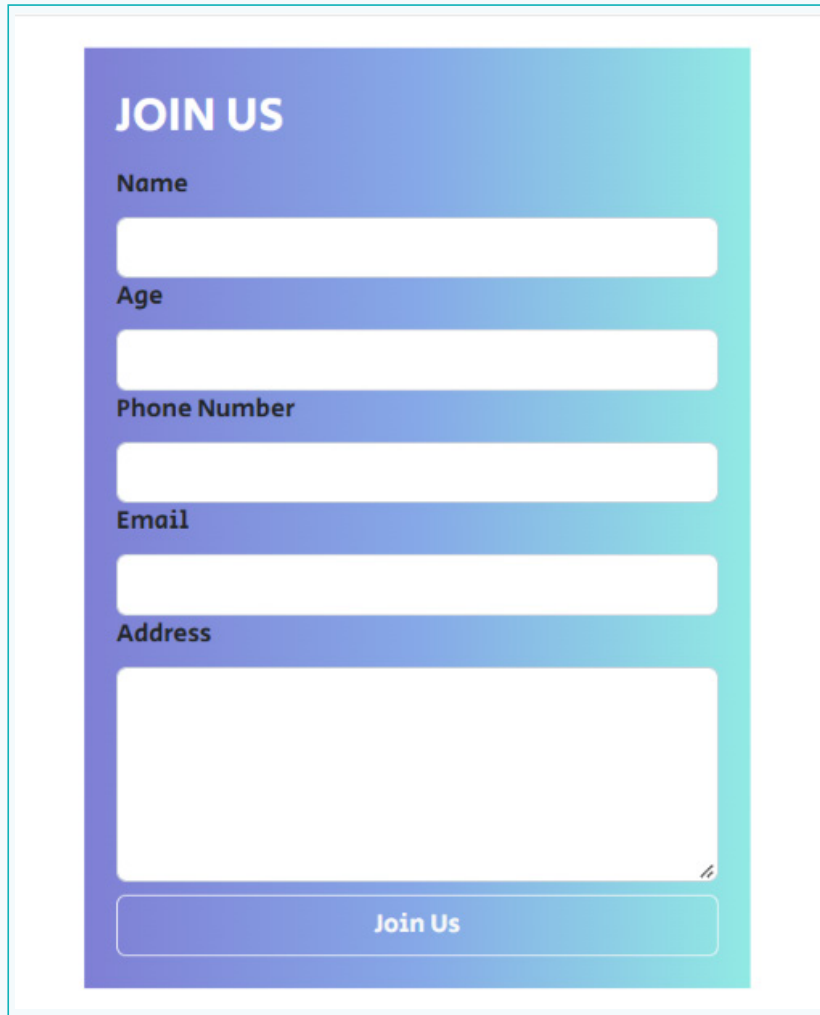
A registration form titled "JOIN US" with a blue-to-teal gradient background. It contains five input fields: "Name", "Age", "Phone Number", "Email", and "Address". The "Address" field is larger than the others. At the bottom is a "Join Us" button.

Figure: Registration Page

To conclude, we created an HTML Form for our animal welfare website and carried out actions that facilitated the storage of user data in a JSON file. We started by learning about HTML forms and the various ways in which they can be used.

We used labels, text boxes, and other elements to create both an HTML page and a form. Following that, we designed the form with Bootstrap and CSS. After that, we looked at how to save user input in a JSON file.

The 'Form Data' that had been stored in the 'Users.json' file was then used to continue our work. Finally, we observed the systematic storage of random data.



### Tech Flash

- **Bootstrap** : A popular front-end framework for building responsive web applications. It provides a set of CSS, JavaScript, and HTML templates that can be used to quickly build a variety of web user interfaces.
- **JSON** : JavaScript Object Notation or JSON, is a lightweight data-interchange format that is easy for humans to read and write, and also easy for machines to parse and generate. Commonly used to exchange data between a server and a web application.